

TomatoGuard AI

Frontend Implementation Specification

React + Vite PWA — page-by-page, matches the Backend document exactly

How to Use This Document

This document assumes the existing agri-smart repo as the starting point — do not rebuild from scratch. Every API call and field name below matches the Backend document exactly; if the two ever disagree, the Backend document is authoritative, since it matches the real database schema.

v1 page/route decisions

Page	v1 status
Landing, Signup, Login, Dashboard, Disease, Soil, Weather, Recommendations, History, Profile	Active — build/rewire per this document
Settings	Active, but frontend-only (localStorage) — no backend table exists for it in v1
Tasks, Schemes, Reports (page)	Cut from v1 — remove their routes from App.jsx and their links from navigation. Keep the .jsx files in src/pages/ (don't delete), just unrouted, so v2 work resumes from what already exists
Admin Login, Admin Dashboard, Admin Farmer Detail, Admin Diseases	NEW — active in v1, mounted separately at /admin/, entirely separate auth from the farmer app. See Section 9

Why unroute instead of delete: these pages already exist as working UI (per your team's own status doc) and your DB already has matching tables. Deleting them would be pure waste — v2 just re-adds the route once Tasks/Schemes/Reports backend work happens.

1. Global changes (apply once, before any page work)

- `src/lib/api.js` — replace/extend with real calls for every endpoint in the Backend document. Keep the existing pattern: every `*_API_URL` comes from a Vite env var, and every call site shows an explicit 'service not connected' state when its URL is unset — this discipline already exists in the repo and must not be removed.
- `src/App.jsx` — Protected route wrapper checks for a real JWT `access_token` in `localStorage` (key: `tomatoTokens`), not any placeholder flag. Remove routes for `/tasks`, `/schemes`, `/reports` per the table above.
- `src/components/AppLayout.jsx` — bottom nav becomes: Dashboard, History, Profile (drop Tasks/Schemes links).
- Remove the `tesseract.js` dependency entirely (`package.json`) — OCR is server-side only now, per the Backend document Phase 4.

PWA setup

- `vite.config.js` — add `vite-plugin-pwa`: `registerType 'autoUpdate'`, `manifest` (name 'TomatoGuard AI', `theme_color` matching brand red, `display 'standalone'`, icons at 192/512/512-maskable), `workbox` precaching for the app shell.
- `src/main.jsx` — call `registerSW()` from `virtual:pwa-register` so the service worker actually activates.
- Disease upload file input uses `capture="environment"` for camera-first behavior on mobile.

2. Phase 1 — Auth Pages

Signup — /signup

Account creation — now includes farm fields since signup auto-creates the farm row server-side.

On-screen elements:

- Fields: full name, email, password, confirm password, phone (optional), village/district/state (optional), farm name, farm location (optional map pin → latitude/longitude)
- 'Create account' button
- Link to Login

API calls:

- POST /auth/signup — send farm_name and lat/lng alongside the user fields in one call, matching the Backend doc's single-transaction signup

Note: Do not add a separate 'create your farm' step — v1 farm creation is invisible to the user, happening inside the same signup call.

Login — /login

Authentication.

On-screen elements:

- Fields: email, password
- 'Login' button
- 'Forgot password' link
- Link to Signup

API calls:

- POST /auth/login
- POST /auth/refresh (silently, when the access token expires)

3. Phase 2 — Dashboard

Dashboard — /dashboard

Central hub — the page a farmer returns to every time.

On-screen elements:

- Summary cards: latest disease result, latest soil analysis, current weather snippet, combined recommendation status
- Primary buttons: 'Scan a leaf', 'Check soil', 'View weather'
- Bottom nav: Dashboard, History, Profile
- Logout

API calls:

- GET /dashboard/summary on load

Note: Each card shows an explicit empty state ('No scan yet — take your first photo') when its field is null — never a fake number, matching the discipline already in the existing codebase.

4. Phase 3 — Disease Detection

Disease — /disease

Upload a tomato leaf photo, get a CNN diagnosis.

On-screen elements:

- 'Upload image' button — file input with capture="environment" (opens camera directly on mobile)
- Image preview with 'Retake/replace'
- 'Analyze' button
- Result card: disease name, confidence %, description, recommended action
- History list below the result (recent scans)

API calls:

- POST /disease/analyze (multipart)
- GET /disease/history

Note: Preserve the existing 'not connected yet' state when VITE_DISEASE_API_URL is unset.

5. Phase 4 — Soil Health

Soil — /soil

Two input paths converging on one result.

On-screen elements:

- Tab toggle: 'Upload report' vs 'Answer questions'
- Upload path: file input → POST /soil/report → editable fields shown (ph/N/P/K/EC/organic matter) → 'Confirm' button
- Questionnaire path: form (crop stage, previous crop, irrigation type, fertilizer used, soil color, drainage condition) → 'Submit'
- Result: fertility rating, fertilizer/irrigation recommendation

API calls:

- POST /soil/report
- POST /soil/report/confirm
- POST /soil/questionnaire
- GET /soil/latest

Note: Do not reintroduce tesseract.js or any client-side OCR — this was explicitly removed. The 'Upload report' path now sends the raw image straight to the backend.

6. Phase 5 — Weather

Weather — /weather

Live and forecast conditions for the farmer's farm location.

On-screen elements:

- Current: temperature, humidity, rainfall, condition icon
- 3-5 day forecast strip
- 'Change location' button (updates the farm's lat/lng via a future PATCH — see Phase 7 Profile)

API calls:

- GET /weather/current
- GET /weather/forecast

Note: If the farm has no saved location, show a clear prompt to set one in Profile rather than an error state.

7. Phase 6 — Recommendations

Recommendations — /recommendations

The combined DSS output — the core value of the platform.

On-screen elements:

- Health banner (good / at-risk / critical, color-coded)
- Sections: fertilizer, irrigation, disease treatment, pest prevention
- 'Back to dashboard' button

API calls:

- GET /recommendations/latest

Note: This is computed fresh by the backend on every load — there's nothing to 'save' client-side. If the endpoint 404s (no data at all yet), show a friendly prompt pointing back to Disease/Soil/Weather rather than a broken page.

8. Phase 7 — History & Profile

History — /history

Chronological log across all data types.

On-screen elements:

- List/table: date, type (scan / soil / weather), quick status
- 'View details' per entry
- Date + type filter

API calls:

- GET /history?type=&from=&to=

Profile — /profile

Account + farm management.

On-screen elements:

- Farm details, location (map pin)
- Change password
- Logout

API calls:

- GET /profile
- PATCH /profile
- POST /auth/change-password

Settings — /settings

Frontend-only in v1 — no backend calls.

On-screen elements:

- Theme toggle (light/dark)
- Notification preference toggle (UI-only, has no effect until Notifications is built in v2)

API calls:

- None — persists to localStorage exactly as it does today

9. Admin Section (separate from the farmer app)

Mounted at `/admin/`. This is a second, independent auth context in the same repo — an admin session and a farmer session must be able to coexist in the same browser without interfering (different `localStorage` key, different route guard, different token audience — see Backend document, Admin Module).

Admin Login — `/admin/login`

Separate login form from the farmer Login page — visually distinct so nobody confuses the two.

On-screen elements:

- Fields: email, password
- 'Login' button — no signup link, no 'forgot password' link (admin accounts are provisioned directly in the database)

API calls:

- POST `/admin/login`

Note: Store the resulting tokens under a different `localStorage` key (`tomatoAdminTokens`) than the farmer app uses (`tomatoTokens`).

Admin Dashboard — `/admin/dashboard`

Farmer list with search.

On-screen elements:

- Search box (name/email/village)
- Paginated table: name, email, village, farm name, active/inactive status, joined date
- Row click → farmer detail

API calls:

- GET `/admin/farmers?search=&offset=&limit=`

Admin Farmer Detail — `/admin/farmers/:id`

One farmer's full picture for oversight.

On-screen elements:

- Profile + farm info
- Recent disease scans, soil analyses, weather records (read-only lists)
- Activate/Deactivate toggle with a confirmation step (this affects a real person's account access)

API calls:

- GET `/admin/farmers/{user_id}`
- PATCH `/admin/farmers/{user_id}/status`

Admin Diseases — `/admin/diseases`

Manage the disease lookup table that powers every farmer's scan results.

On-screen elements:

- Table of all disease entries

- 'Add disease' form (name, description, symptoms, causes, prevention, treatment)
- Edit-in-place or edit form per row

API calls:

- GET /admin/diseases
- POST /admin/diseases
- PATCH /admin/diseases/{disease_id}

Note: This table's content appears directly on the farmer-facing Disease Detection results page — treat copy quality here as farmer-facing, not just internal notes.

- AdminProtected route wrapper — same pattern as the farmer Protected wrapper, but checks tomatoAdminTokens and redirects to /admin/login, never to the farmer /login.
- No shared layout/nav component between admin and farmer sections — keep them visually and structurally separate so there's no risk of a farmer stumbling into /admin/* through a shared nav link.

10. Acceptance Criteria (whole frontend, v1)

- Every page's API calls use the exact endpoint paths and field names from the Backend document — no invented fields.
- Every not-yet-connected service shows an explicit empty/pending state, never fabricated data.
- /tasks, /schemes, /reports routes are removed from navigation and routing; their page files remain in the repo unrouted.
- App installs as a PWA (manifest + service worker verified via Lighthouse).
- Mobile-first layout holds at 375px width on every active page.
- An admin session and a farmer session can be open in the same browser simultaneously without either logging the other out.
- Navigating to /admin/* with a farmer token (or no token) redirects to /admin/login, never shows admin content.